requestAnimationFrame

> "request animation frame" 请求动画帧

从名字上看我们就能看出它是一个专门用于做动画的API，使用方法如下：

```
// 返回一个唯一标识用于取消执行
const requestID = window.requestAnimationFrame((time) => {
  // 回调操作
});

// 取消执行
window.cancelAnimationFrame(requestID);
```

它的回调会传入一个时间值，这个时间值并不是系统时间，而是当前页面自加载以来的运行时间。

> 这个时间值和 `performance.now()` 返回值是一样的，都是当前页面自加载以来的运行时间。
>
> 比起用 `console.time()` 计算脚本过程运行时间， `performance.now()` 可以返回时间值用于计算以实现更多处理，且都具有良好的时间精度，可以达到微秒级。
>
> 如果使用 `Date.now()` ，时间精度不及 `performance.now()` ，是毫秒级精度。

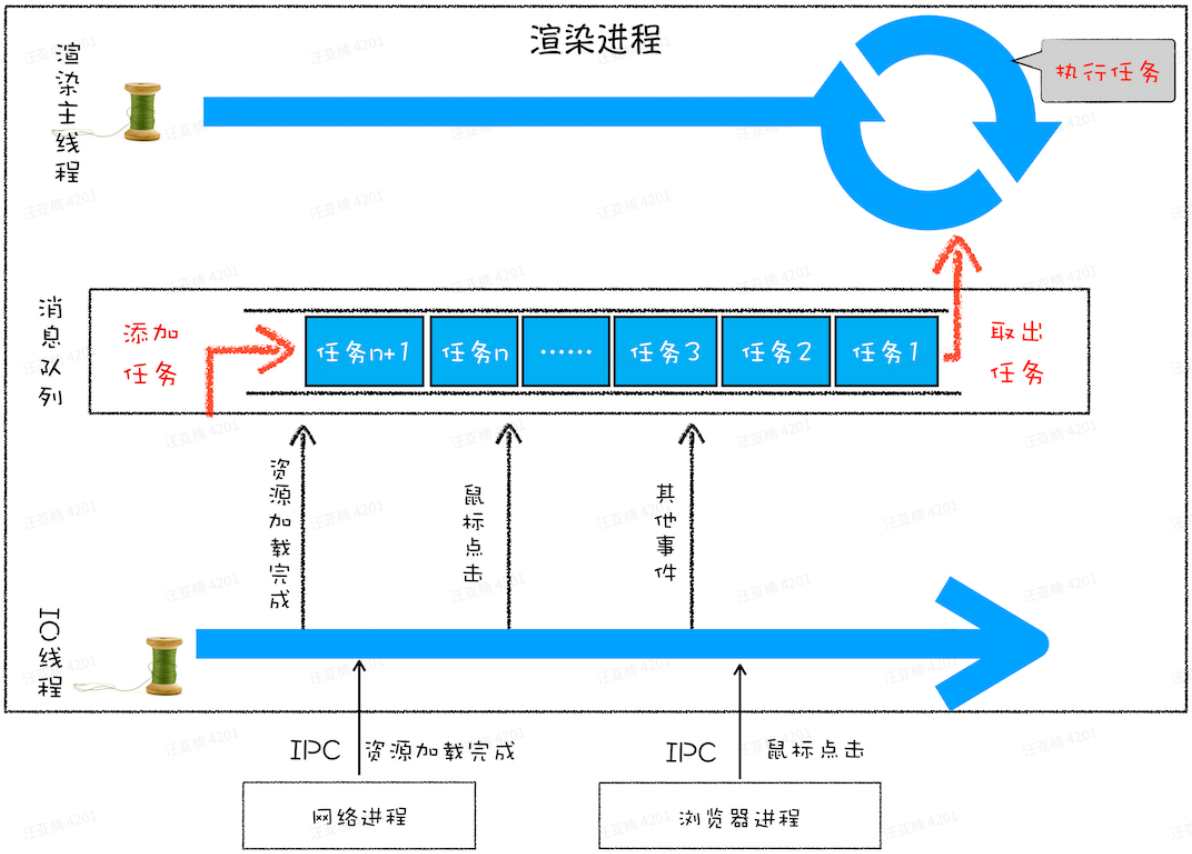
使用知道了，那如何理解它在动画操作中的专业度呢？我们从它的诞生开始了解。

1）诞生的背景

假如没有它，我们使用 `JavaScript` 做动画无非是使用 `setInterval` ：

```
// 每延迟delayTime秒执行一次回调
setInterval(callback, delayTime);
```

我们知道浏览器是由消息队列和事件循环系统驱动的，当一个延迟任务创建后它会进入延时队列（另一个消息队列），待当前消息队列的任务执行完后会开始计算出延时队列中所有已经到期的任务并执行，然后循环往复。



在这里我们很容易找到一个很关键的点："**待当前消息队列的任务执行完...**"。

如果当前消息队列中的任务执行时间过长就会导致这个延时任务没办法在指定的延迟时间后准点执行，所以 `setInterval` 的时效性是很差的，这也导致拿 `setInterval` 做动画会出现"抽搐感"——每两帧的间隔时间不一样。

时效性是指一个事件或行为的有效期限，即在一定的时间内必须完成某项任务，否则就会失去有效性。

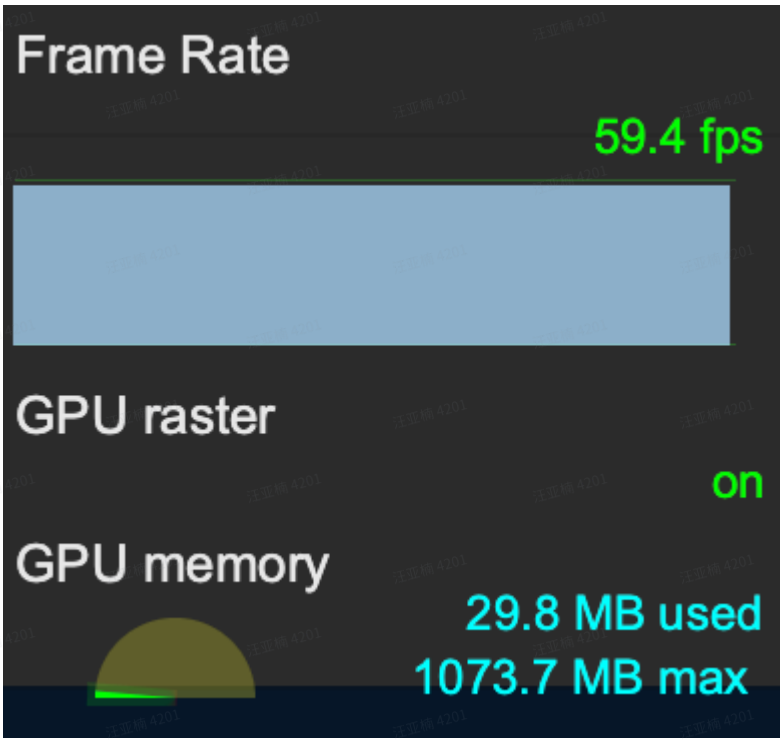
既然旧方法行不通，那就必然会诞生新方法。

2）“浏览器重绘我就执行”

一般显示屏一秒钟刷新的频率是60次（60HZ），也就是约17毫米刷新一次。

同样的，对于浏览器而言，如果绘制频率能和显示屏刷新频率保持一致，每秒绘制60次那能达到最好的页面动画效果，这里的每秒绘制多少次指的也就是我们常说的 `FPS——每秒传输帧数(Frames Per Second)` 。

在 `Chrome` 中我们可以通过运行 `fps` 命令打开浏览器的帧数计算器。



在使用 `setInterval` 实现动画时，我们也会设置延时时间为16 ~ 17毫秒，但是因为延时器的时效性差，我们很难得到一个平滑过渡的动画效果。

为了解决这个问题， `requestAnimationFrame` 就诞生了，它将回调执行的时机放在浏览器重绘前的那一刻，这样就能保持和浏览器的绘制频率一致，得到一个更好的平滑动画效果。

当然如果有任务卡着阻塞浏览器的重绘，也会导致该回调的执行延迟。

`requestAnimationFrame` 是个一次性的执行方法，并不像 `setInterval` 那样自动创建下一次的执行任务，所以需要在回调内递归执行该方法。

思考：它能替代节流使用，这是正确的吗？

网上有相当一部分人说：由于 `requestAnimationFrame` 的执行和浏览器重绘时机保持一致，当监听页面 `resize` 、 `scroll` 事件时，可以用它替代 `throttle` 的使用。

我的看法：是错误的。

`requestAnimationFrame` 并不是同一时间只有一个回调会生效，也就是连续调用时，并不是后者替换前者，而是它们会同时等待下一次重绘前执行，最后他们都会执行且回调中的时间值也是一样的。

```
window.requestAnimationFrame((time) => {
  console.log(time)
})
window.requestAnimationFrame((time) => {
  console.log(time)
```

```
})
```

所以如果你想通过以下写法替代节流：

```
window.onresize = () => {
  console.count('onresize');
  window.requestAnimationFrame(() => {
    console.count('request');
  });
};
```

那是做不到的。

```
onresize: 13
request: 13
onresize: 14
request: 14
onresize: 15
request: 15
```

3）如何手动实现FPS的计算

https://codepen.io/huanjinliu/embed/gOQoYrw?editors=1011